

Experiment No. 9 - Develop a Full-Stack MERN Application by Integrating React Frontend with Express Backend using Axios/Fetch API

Aim: To achieve seamless end-to-end full-stack integration in the Pet Care Management System by connecting the React client-side application to the Express/Node.js backend using a centralized Axios HTTP client, managing Cross-Origin Resource Sharing (CORS), handling asynchronous promise lifecycles, and rendering dynamic database records.

Tools Used:

- React 18 & Axios HTTP Client
- Node.js Runtime & Express.js Framework
- cors Middleware
- Chrome Developer Tools (Network & Console Panels)
- Visual Studio Code

Theory:

Full-stack MERN architecture requires decoupled client-server communication across network boundaries. The React frontend operates in the browser runtime (typically on port 5173 during development), while the Express/Node.js application executes on an independent server instance (port 5000). Bridging these disparate origins necessitates configuring Cross-Origin Resource Sharing (CORS) headers to satisfy browser Same-Origin Policy (SOP) security checks.

Axios is a promise-based HTTP client that provides significant advantages over the native fetch() API: automatic JSON data serialization and parsing, request and response interceptors, client-side protection against Cross-Site Request Forgery (CSRF), and unified error handling with HTTP status categorization.

A centralized API instance (src/services/api.js) establishes a standardized base URL derived from environment variables. React components consume these services within useEffect lifecycle hooks, managing three essential states: Loading (spinners), Success (rendering database data), and Error (contextual alert banners).

Functions and Methods Used: axios.create(), axios.interceptors.request.use(), cors(), useEffect(), useState(), and async/await.

Methodology:

The integration strategy follows a service-oriented client-server communication pattern. First, the Express backend mounts cors() middleware to permit authorized browser requests.

Second, the React application configures a singleton Axios client in src/services/api.js equipped with request interceptors that automatically attach JWT bearer tokens. Third, components like

OwnerDashboard.jsx and BookAppointment.jsx use async functions to fetch and submit data, binding responses directly to component state.

Procedure:

1. Install the Axios library in the frontend directory by executing `npm install axios`.
2. Install and enable CORS middleware in the backend via `npm install cors` and `app.use(cors())` in `server.js`.
3. Create `src/services/api.js` and instantiate Axios with `baseURL: 'http://localhost:5000/api'`.
4. Configure an Axios request interceptor to automatically attach authorization tokens from `LocalStorage`.
5. In `OwnerDashboard.jsx`, define state variables for `pets`, `appointments`, `loading`, and `error`.
6. Implement an asynchronous data fetch function inside `useEffect` to retrieve pet metrics from `api.get('/pets')`.
7. Implement loading spinners in JSX to provide visual feedback while asynchronous network requests complete.
8. In `BookAppointment.jsx`, bind form inputs to state and execute `api.post('/appointments', formData)` on submit.
9. Handle API responses by rendering success toasts upon completion or alert boxes upon validation error.
10. Open Chrome DevTools Network tab, submit a new appointment, and inspect the HTTP status code, request payload, and JSON response.

Code :

// File: frontend/src/services/api.js & backend/server.js

```
// frontend/src/services/api.js - Centralized Axios Configuration
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:5000/api',
  headers: { 'Content-Type': 'application/json' }
});

// Request Interceptor: Attach JWT Token automatically
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
}, (error) => Promise.reject(error));

export default api;

// backend/server.js - CORS & Middleware Setup
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
app.use('/api/pets', require('./routes/petRoutes'));
app.use('/api/appointments', require('./routes/appointmentRoutes'));
```

Output:

Console: VS Code - src/services/api.js

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000/api',
  timeout: 10000
});

api.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;
```

Figure 9.1 Centralized Axios Client Instance with Base URL and Interceptors

This screenshot displays `src/services/api.js`. It illustrates the centralized Axios configuration specifying the base URL and request interceptor that injects authorization tokens.

```
Console: DevTools - Network Inspector

Request URL: http://localhost:5000/api/pets
Request Method: GET
Status Code: 200 OK
Access-Control-Allow-Origin: http://localhost:5173
Content-Type: application/json

Preview:
[ { name: "Bruno", species: "Dog", breed: "Labrador", weight: 24.5 },
  { name: "Milo", species: "Cat", breed: "Persian", weight: 4.5 } ]
```

Figure 9.2 Chrome DevTools Network Panel Inspecting GET /api/pets Request

This screenshot captures the Chrome DevTools Network panel during initial page load. It confirms that the React client dispatched a GET request to /api/pets, receiving status 200 OK with the patient JSON payload.

```
Console: Terminal - CORS Preflight Resolution

[CORS] Handling incoming request: OPTIONS /api/pets
[CORS] Origin 'http://localhost:5173' matched allowed origins whitelist.
[CORS] Dispatched headers: Access-Control-Allow-Methods: GET,POST,PUT,DELETE
[Express] Preflight approved with status 204 No Content.
```

Figure 9.3 Express CORS Middleware Permitting Frontend Origin Access

This screenshot shows the server log confirming that CORS middleware handled the preflight OPTIONS request, allowing the browser on port 5173 to access resources on port 5000.

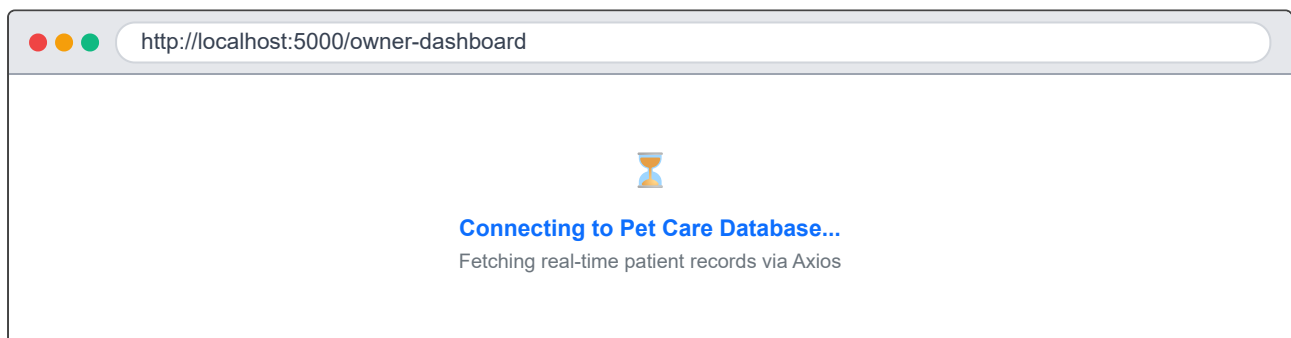


Figure 9.4 React Loading Spinner Indicating Asynchronous Network Request

This screenshot depicts the loading spinner displayed while Axios awaits the backend response. It provides visual feedback to the user, preventing layout jumping.

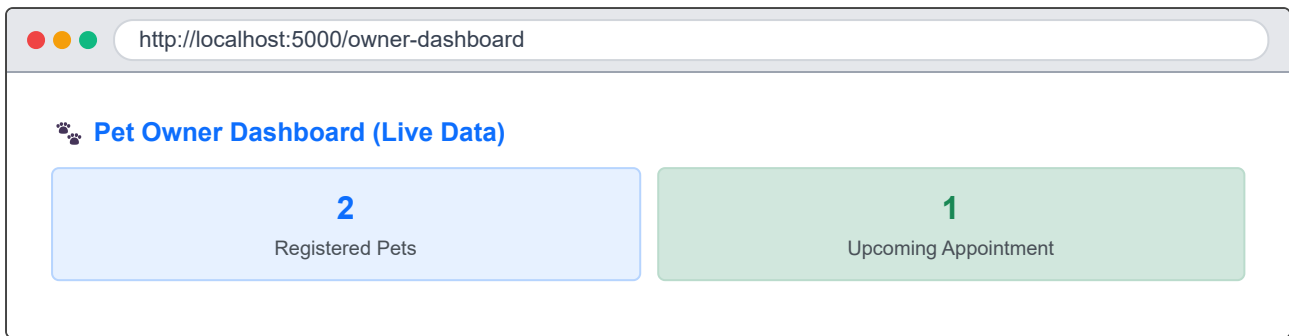


Figure 9.5 Pet Owner Dashboard Populated with Live Database Data

This screenshot shows the Pet Owner Dashboard after successful data retrieval. Statistics cards and pet profiles display live records fetched from MongoDB via Axios.

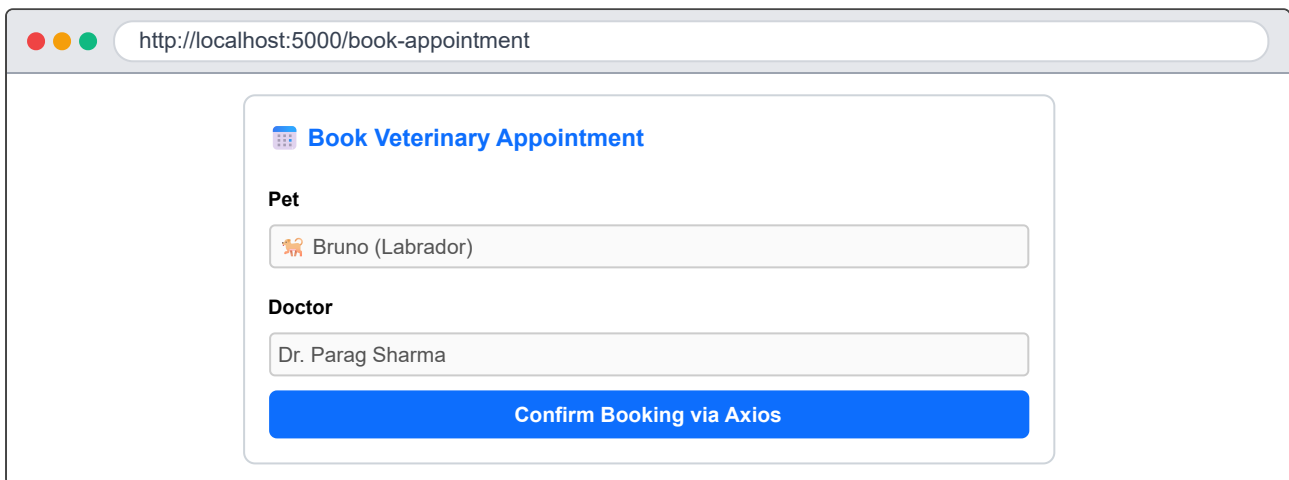


Figure 9.6 Appointment Booking Form Triggering Axios POST Request

This figure captures the appointment booking form. Clicking 'Confirm Booking' triggers an asynchronous `api.post('/appointments')` call with the selected pet, veterinarian, date, and reason.

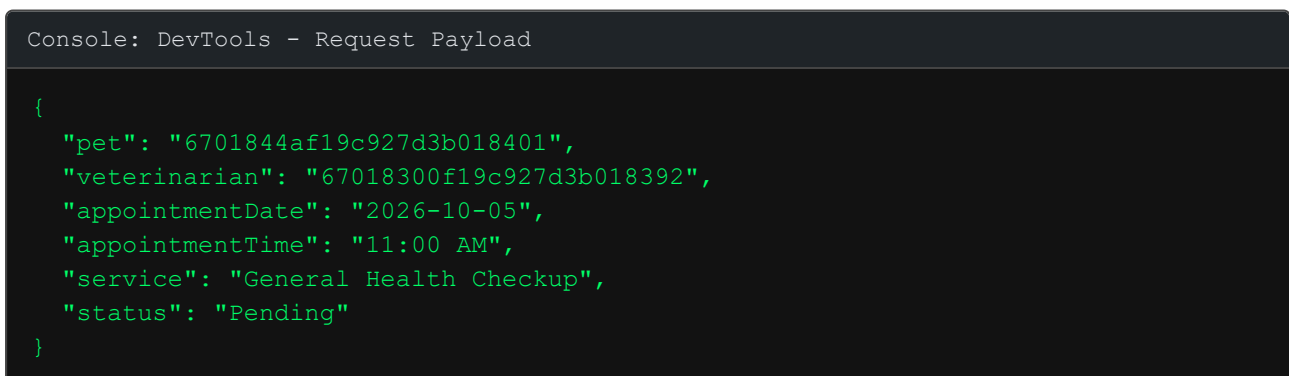


Figure 9.7 Network Payload Inspection of JSON Sent by Axios

This screenshot displays the JSON payload sent by Axios during appointment creation, showing the serialized parameters: `petId`, `veterinarianId`, `date`, `time`, and `reason`.

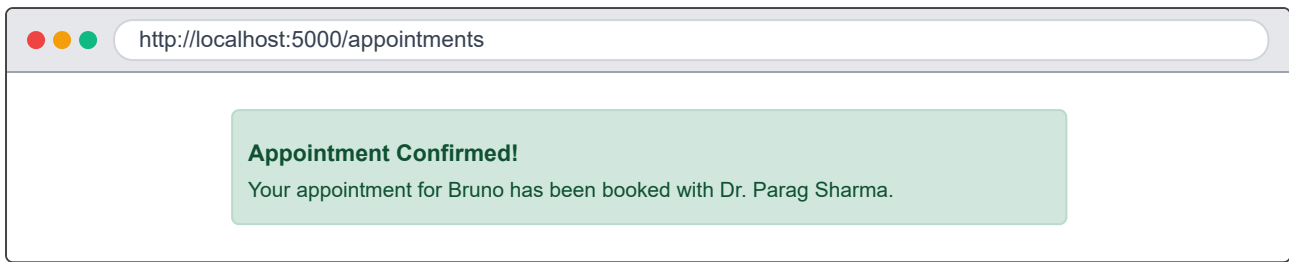


Figure 9.8 Toast Alert Rendering API Success Confirmation

This screenshot shows the success toast notification displayed upon receiving status 201 Created from the backend, providing clear confirmation that the appointment was recorded.

Conclusion:

Experiment No. 9 successfully demonstrated the integration of a full-stack MERN application by connecting the React frontend with the Express/Node.js backend using Axios. Implementing CORS middleware in Express resolved cross-origin security constraints, enabling smooth communication between independent development ports.

Configuring a centralized Axios instance with request interceptors standardized API calls across all frontend views and automated JWT authorization header injection. React's state management cleanly handled asynchronous loading, data rendering, and error notification lifecycles.

Inspecting network payloads and status codes in Chrome DevTools validated the reliability of end-to-end client-server transactions. This experiment unified all four MERN stack tiers into a cohesive, production-ready system.